FACULTAD DE INGENIERÍA Y ARQUITECTURA

Carrera Profesional de Ingeniería de Computación y Sistemas

Arquitectura de Software para Sistemas de Información

Título: Diseño arquitectónico y prueba de concepto de EduTrack LMS, ecosistema de gestión de aprendizaje

Ciclo: VII

Autores:

Arestegui Parra Milguar Esmith
Flores Aguilar Erick Leonardo
Mamani Acuña Tafarel Aguinaga
Pinto Villavicencio Angeles

Docente: Mg. Aníbal Sardón Paniagua

Arequipa - Perú

2026

ÍNDICE

CAPÍTULO 1 – ASPECTOS GENERALES....3
1.1 CONTEXTO Y PROBLEMÁTICA....3
1.2 JUSTIFICACIÓN....3
1.3 DEFINICIÓN DE OBJETIVOS....3
1.3.1 OBJETIVO GENERAL....3
1.3.2 OBJETIVOS ESPECÍFICOS....4
1.3.3 ALCANCES Y LIMITACIONES....4
CAPÍTULO 2 – VISIÓN Y ALCANCE....6
2.1 VISIÓN DE LA SOLUCIÓN....6
2.1.1 OBJETIVOS DEL NEGOCIO....6
2.1.2 CARACTERÍSTICAS DEL SISTEMA....6
2.2 DRIVERS ARQUITECTÓNICOS....7
2.2.1 DRIVERS FUNCIONALES....7
2.2.2 DRIVERS DE ATRIBUTOS DE CALIDAD....8
2.2.3 DRIVERS DE RESTRICCIONES....10
CAPÍTULO 3 – DISEÑO DEL SISTEMA....11
3.1 MODELO 4+1....11
3.1.1 VISTA DE ESCENARIOS....11
3.1.2 VISTA LÓGICA....12
3.1.3 VISTA DE PROCESOS....16
3.1.4 VISTA DE DESARROLLO....19
3.1.5 VISTA FÍSICA....21
3.2 MODELO C4....23
3.2.1 DIAGRAMA DE CONTEXTO....23
3.2.2 DIAGRAMA DE CONTENEDORES....24
3.2.3 DIAGRAMA DE COMPONENTES....24
3.2.4 DISEÑO DE LA BASE DE DATOS....26
3.3 REGISTRO DE DECISIONES ARQUITECTÓNICAS (ADR)....30
3.4 PATRONES DE DISEÑO Y PRINCIPIOS APLICADOS....33
3.5 PRUEBA DE CONCEPTO, PRUEBAS Y GESTIÓN DEL PROYECTO....34
3.5.1 ALCANCE DE LA PRUEBA DE CONCEPTO....34
3.5.2 ESTRATEGIA DE PRUEBAS....34
3.5.3 GESTIÓN CON SCRUM Y GITHUB....34
CONCLUSIONES....36
RECOMENDACIONES....36
BIBLIOGRAFÍA....37

CAPÍTULO 1
ASPECTOS GENERALES

1.1 CONTEXTO Y PROBLEMÁTICA

La institución educativa objeto de este trabajo enfrenta un reto de escalabilidad en su gestión académica digital. Su sistema actual de distribución de materiales está fragmentado, el seguimiento del estudiante se realiza de forma manual y no existe un mecanismo automatizado para evaluar, calificar y detectar a tiempo a los estudiantes en riesgo de desaprobación o deserción.

Se requiere desarrollar **EduTrack LMS**, una plataforma centralizada de gestión de aprendizaje (Learning Management System) que soporte modalidades síncronas y asíncronas, que gestione el ciclo de vida completo del contenido pedagógico, automatice las evaluaciones y fomente la interacción mediante foros, garantizando alta disponibilidad durante los periodos críticos de exámenes.

Los principales problemas identificados son los siguientes:

- Materiales distribuidos en canales dispersos, sin versionado ni control de publicación.
- Evaluaciones y calificaciones procesadas de forma manual, con riesgo de errores y sin trazabilidad de los cambios de nota.
- Riesgo de pérdida de respuestas y caídas del servicio cuando muchos estudiantes rinden un examen simultáneamente.
- Dependencia de un Sistema de Registro Académico legado del cual deben obtenerse las matrículas, sin una integración confiable.
- Ausencia de analítica para identificar tempranamente a estudiantes en riesgo.

1.2 JUSTIFICACIÓN

Más allá de almacenar archivos, un LMS institucional es un sistema de información crítico: concentra evaluaciones con consecuencias académicas y debe responder bien bajo picos de carga muy concentrados en el tiempo. Por ello, las decisiones de arquitectura (cómo se aísla el dominio pedagógico, cómo se manejan los exámenes concurrentes, cómo se integra el sistema legado y cómo se audita cada cambio de nota) condicionan el éxito del proyecto y deben tomarse de forma explícita y justificada antes de la construcción.

1.3 DEFINICIÓN DE OBJETIVOS

1.3.1 OBJETIVO GENERAL

Diseñar la arquitectura de software de EduTrack LMS y construir una prueba de concepto de sus módulos críticos, de modo que la institución disponga de una plataforma centralizada, escalable y trazable para gestionar el ciclo de vida de los cursos, automatizar las evaluaciones y monitorear el progreso de los estudiantes, con alta disponibilidad en los periodos de exámenes.

1.3.2 OBJETIVOS ESPECÍFICOS

- Identificar y priorizar los drivers arquitectónicos (funcionales, atributos de calidad y restricciones) del sistema a partir del caso de estudio.
- Seleccionar y justificar los estilos y patrones arquitectónicos (Arquitectura Hexagonal con núcleo de dominio aislado) que satisfagan los drivers de mayor prioridad.
- Modelar la arquitectura mediante el Modelo 4+1 vistas y el Modelo C4 (contexto, contenedores y componentes), evidenciando las integraciones externas.
- Diseñar una estrategia de concurrencia para el módulo de exámenes en línea que evite bloqueos de base de datos y la pérdida de respuestas parciales.
- Garantizar la trazabilidad de las calificaciones mediante un registro de auditoría inmutable.
- Documentar las decisiones arquitectónicas (ADR) y los patrones de diseño GoF/GRASP y principios SOLID aplicados, y definir el alcance de la prueba de concepto en Java con Spring Boot y PostgreSQL.

1.3.3 ALCANCES Y LIMITACIONES

1.3.3.1 Dentro del Alcance

Tabla 1. *Módulos y submódulos dentro del alcance*

Módulo	Submódulos y descripción
Gestión del ciclo de vida del curso	Creación del sílabo, estructuración modular (unidades, sesiones), publicación de material multimedia, versionado de contenidos y flujo de aprobación (Borrador, En revisión, Publicado, Archivado).
Evaluación y calificación	Banco de preguntas, cuestionarios aleatorios, exámenes contra reloj, recepción de tareas, calificación automática (opción múltiple) y manual (rúbricas), promedio ponderado.
Interacción y colaboración	Foros por curso o unidad, hilos de debate y sistema de notificaciones (respuestas y fechas de entrega próximas).
Seguimiento y analítica	Porcentaje de completitud, tiempos de conexión e identificación temprana de estudiantes en riesgo.
Integraciones	Sincronización nocturna de matrículas con el Sistema de Registro Académico (servicio legado SOAP/REST) e integración con una API externa de detección de plagio.

Módulo	Submódulos y descripción
Seguridad, auditoría y administración	Autenticación y autorización por rol (Estudiante, Docente, Diseñador Instruccional, Administrador), registro de auditoría inmutable y monitoreo del rendimiento.

1.3.3.2 Fuera del Alcance

- Gestión de matrículas, pagos y registro de notas oficiales: pertenecen al Sistema de Registro Académico, del cual EduTrack solo consume la lista de inscritos.
- Desarrollo de un motor propio de detección de plagio: se integra una API externa.
- Plataforma propia de videoconferencia: las sesiones síncronas se enlazan mediante herramientas externas.
- Aplicaciones móviles nativas: la interfaz web será responsiva.
- Analítica avanzada con modelos de aprendizaje automático: se parte de reglas de riesgo configurables.

1.3.3.3 Limitaciones y supuestos

Las cifras de carga (por ejemplo, 3 000 estudiantes concurrentes en un examen) son supuestos de diseño que deben validarse con la institución. La prueba de concepto se limita a los módulos críticos (Motor de Evaluaciones e Integración de Matrículas) y se despliega en un entorno de pruebas en la nube.

CAPÍTULO 2
VISIÓN Y ALCANCE

2.1 VISIÓN DE LA SOLUCIÓN

EduTrack LMS será una plataforma moderna de gestión de aprendizaje, accesible desde navegador web, que centralizará la publicación de contenidos, la evaluación y la interacción entre estudiantes y docentes. Consumirá las matrículas desde el Sistema de Registro Académico y se apoyará en servicios externos para la detección de plagio y el envío de notificaciones.

2.1.1 OBJETIVOS DEL NEGOCIO

Tabla 2. *Objetivos de negocio*

ID	Descripción del objetivo de negocio
ON-1	Centralizar la distribución de materiales y la gestión de cursos en una única plataforma, reemplazando el esquema fragmentado actual.
ON-2	Automatizar las evaluaciones y reducir el esfuerzo manual de calificación, con retroalimentación inmediata en preguntas de opción múltiple.
ON-3	Garantizar alta disponibilidad y rendimiento durante los periodos críticos de exámenes.
ON-4	Fomentar la interacción y colaboración mediante foros y notificaciones oportunas.
ON-5	Disponer de seguimiento y analítica para la identificación temprana de estudiantes en riesgo de deserción o desaprobación.
ON-6	Asegurar la trazabilidad de los cambios de calificación y de los exámenes publicados.

2.1.2 CARACTERÍSTICAS DEL SISTEMA

Tabla 3. *Características del sistema (requisitos funcionales y no funcionales)*

ID	Descripción	Tipo	Prioridad	Objetivo asociado
CAR-01	El sistema debe permitir crear el sílabo y estructurar el curso en módulos, sesiones y materiales.	RF	Alta	ON-1
CAR-02	El sistema debe permitir publicar material multimedia con versionado de contenidos.	RF	Alta	ON-1
CAR-03	El sistema debe exigir la aprobación del Diseñador Instruccional antes de que un curso sea visible para los estudiantes.	RF	Alta	ON-1
CAR-04	El sistema debe generar cuestionarios aleatorios a partir de un banco de preguntas y permitir rendirlos contra reloj.	RF	Alta	ON-2
CAR-05	El sistema debe permitir la entrega de tareas (archivos) por parte de los estudiantes.	RF	Alta	ON-2
CAR-06	El sistema debe calificar automáticamente (opción múltiple) y manualmente (rúbricas) y calcular promedios ponderados.	RF	Alta	ON-2

ID	Descripción	Tipo	Prioridad	Objetivo asociado
CAR-07	El sistema debe ofrecer foros por curso o unidad, hilos de debate y notificaciones de respuestas y fechas de entrega.	RF	Media	ON-4
CAR-08	El sistema debe monitorear el progreso, los tiempos de conexión e identificar estudiantes en riesgo.	RF	Media	ON-5
CAR-09	El sistema debe sincronizar cada madrugada la lista de estudiantes inscritos por curso desde el Sistema de Registro Académico.	RF	Alta	ON-1
CAR-10	El sistema debe integrarse con una API externa de detección de plagio para los trabajos subidos.	RF	Media	ON-2
CAR-11	El módulo de exámenes debe soportar picos masivos de transacciones concurrentes sin bloqueo de base de datos y sin perder respuestas parciales ante una desconexión temporal.	RNF	Alta	ON-3
CAR-12	Todo cambio de calificación o modificación de un examen ya publicado debe generar un registro de auditoría inmutable.	RNF	Alta	ON-6
CAR-13	El sistema debe administrar roles de usuario (Estudiante, Docente, Diseñador Instruccional, Administrador).	RF	Alta	ON-1
CAR-14	El sistema debe permitir al administrador monitorear el rendimiento de la aplicación.	RNF	Media	ON-3

Tabla 4. *Planificación del alcance por entregas*

Entrega	Tema principal	Características a incluir
1.0	Núcleo académico: cursos, evaluaciones y matrículas	CAR-01, CAR-02, CAR-03, CAR-04, CAR-06, CAR-09, CAR-13
2.0	Concurrencia, trazabilidad e integraciones	CAR-05, CAR-10, CAR-11, CAR-12
3.0	Interacción y analítica	CAR-07, CAR-08, CAR-14

2.2 DRIVERS ARQUITECTÓNICOS

Los drivers arquitectónicos son los requisitos que influyen directamente en las decisiones de arquitectura: los casos de uso primarios, los escenarios de atributos de calidad más relevantes y las restricciones impuestas por el cliente y por la organización de desarrollo.

2.2.1 DRIVERS FUNCIONALES

Los requisitos funcionales que especifican las interacciones entre los usuarios y el sistema se expresan mediante casos de uso (Cockburn, 2001), derivados de las características del sistema. El modelo UML correspondiente se presenta en la sección 3.1.1.

Tabla 5. *Casos de uso del sistema*

ID	Descripción	Característica asociada
CU-01	Permite consultar los cursos matriculados y el avance del estudiante.	CAR-01, CAR-08

ID	Descripción	Característica asociada
CU-02	Permite consumir el material multimedia publicado en las sesiones del curso.	CAR-02
CU-03	Permite rendir una evaluación en línea contra reloj, con autoguardado de respuestas.	CAR-04, CAR-11
CU-04	Permite entregar una tarea (archivo) que será verificada contra plagio.	CAR-05, CAR-10
CU-05	Permite participar en foros: crear hilos, responder y suscribirse.	CAR-07
CU-06	Permite diseñar la estructura del curso (módulos, sesiones, materiales) y versionar contenidos.	CAR-01, CAR-02
CU-07	Permite redactar y administrar el banco de preguntas.	CAR-04
CU-08	Permite calificar entregables de forma manual mediante rúbricas y registrar la nota.	CAR-06, CAR-12
CU-09	Permite moderar los foros del curso.	CAR-07
CU-10	Permite revisar y aprobar la estructura y el contenido de un curso antes de su publicación.	CAR-03
CU-11	Permite monitorear el rendimiento académico, la conexión y el riesgo de deserción.	CAR-08, CAR-14
CU-12	Permite sincronizar las matrículas desde el Sistema de Registro Académico.	CAR-09
CU-13	Permite administrar roles y usuarios del sistema.	CAR-13
CU-14	Permite consultar el registro de auditoría de calificaciones y exámenes.	CAR-12
CU-15	Permite verificar el plagio de un trabajo mediante la API externa.	CAR-10

Tabla 6. *Casos de uso primarios y criterios de elección*

ID	Criterios de elección (justificación)
CU-03	Es el flujo con mayor exigencia de concurrencia y disponibilidad del sistema (CAR-11, ON-3); condiciona el uso de buffer de sesiones, bloqueo optimista y procesamiento asíncrono.
CU-08	Toda calificación debe ser trazable (CAR-12, ON-6); condiciona el diseño del registro de auditoría inmutable y el uso del patrón Strategy para los algoritmos de calificación.
CU-12	Es la integración crítica con el sistema legado (CAR-09); condiciona el aislamiento del dominio mediante puertos y adaptadores y la sincronización idempotente con reintentos.
CU-04	Integra una API externa y archivos de gran tamaño (CAR-10); condiciona la comunicación asíncrona mediante cola de mensajes.

2.2.2 DRIVERS DE ATRIBUTOS DE CALIDAD

Los atributos de calidad relevantes se identificaron mediante la técnica de escenarios y se priorizaron considerando dos dimensiones con valores alto (A), medio (M) y bajo (B): la importancia para el cliente o negocio y la dificultad de implementación. De acuerdo con la priorización, los atributos más relevantes son EAC-01, EAC-02 y EAC-04. El primer escenario se desglosa en sus seis partes y los demás se describen de forma directa.

Tabla 7. *Escenario EAC-01 desglosado en sus seis partes*

Parte	Descripción
Fuente de estímulo	Estudiantes que rinden un examen en línea (se asume un pico de 3 000 concurrentes).
Estímulo	Envían respuestas parciales (autoguardado) y finalizan el examen en una misma ventana de tiempo.
Artefacto	Módulo de exámenes en línea (CU-03).
Entorno	Periodo de exámenes, operación con carga pico.
Respuesta	El sistema registra cada respuesta sin bloqueo de base de datos y sin perder información.
Medida de la respuesta	El 95% de las operaciones de guardado responde en un tiempo no mayor a un segundo y no se pierde ninguna respuesta confirmada al estudiante.

Tabla 8. *Escenarios de atributos de calidad y su priorización*

ID	Categoría	Escenario	Prioridad (negocio, dificultad)
EAC-01	Desempeño y concurrencia	Ver Tabla anterior: pico de estudiantes guardando respuestas durante un examen.	(A, A)
EAC-02	Disponibilidad	Falla una instancia de la aplicación durante un examen. El sistema continúa atendiendo con las demás réplicas y recupera la capacidad total en un tiempo no mayor a cinco minutos; la disponibilidad objetivo en ventanas de examen es de 99,5%.	(A, M)
EAC-03	Tolerancia a fallos de conexión	Un estudiante pierde la conexión durante un examen. Al reconectarse retoma el intento en curso y conserva sus respuestas hasta el último autoguardado (máximo 15 segundos de antigüedad), sin duplicados.	(A, M)
EAC-04	Trazabilidad	Un docente modifica la calificación de un estudiante en operación normal. El sistema genera un registro inmutable con usuario, fecha y hora, valor anterior y valor nuevo; el intento de modificar o borrar dicho registro es rechazado.	(A, M)
EAC-05	Seguridad	Un estudiante intenta acceder a un curso en estado Borrador o a datos de otra sección. El sistema deniega el acceso según su rol; toda comunicación viaja cifrada (TLS 1.2 o superior).	(A, B)
EAC-06	Interoperabilidad	El servicio legado de matrículas no responde durante la sincronización nocturna. El sistema reintenta hasta tres veces, registra el fallo y alerta al administrador sin afectar la operación del LMS.	(M, M)
EAC-07	Modificabilidad	Un desarrollador reemplaza el proveedor de detección de plagio o agrega un nuevo tipo de evaluación en tiempo de desarrollo. El cambio se realiza sin modificar el núcleo de dominio.	(M, M)
EAC-08	Facilidad de prueba	Un probador ejecuta pruebas unitarias de las reglas de calificación y de aprobación sin necesidad de base de datos ni servicios externos; el dominio alcanza una cobertura de pruebas mínima de 80%.	(M, B)

2.2.3 DRIVERS DE RESTRICCIONES

Tabla 9. *Restricciones del sistema*

ID	Tipo	Restricción
RES-01	Del cliente	El LMS no gestiona matrículas: debe consumir un servicio web legado (SOAP o REST) del Sistema de Registro Académico para sincronizar la lista de inscritos por curso cada madrugada.
RES-02	Del cliente	Se debe integrar una API externa de detección de plagio para los trabajos subidos.
RES-03	Del cliente	Las modificaciones de calificaciones y de exámenes publicados deben generar un registro de auditoría inmutable.
RES-04	Tecnológica	Back-end en un ecosistema orientado a objetos robusto (Java con Spring Boot), persistencia relacional en PostgreSQL y front-end con HTML, CSS y JavaScript.
RES-05	Tecnológica	Aplicación de principios SOLID, patrones de diseño apropiados y separación de responsabilidades (DTOs para no exponer entidades de base de datos).
RES-06	De la organización de desarrollo	Equipo de al menos tres integrantes, repositorio en GitHub con metodología SCRUM y requisitos funcionales y no funcionales reflejados como historias de usuario en GitHub Projects.
RES-07	De la organización de desarrollo	Uso de tecnologías en la nube y pruebas de software como parte de los entregables.
RES-08	Entorno de operación	El sistema se usará desde navegadores web modernos (Chrome, Firefox, Edge y Safari en sus versiones vigentes) con diseño responsivo; no se usarán Flash ni applets.

CAPÍTULO 3
DISEÑO DEL SISTEMA

Los drivers del capítulo anterior conducen a una **Arquitectura Hexagonal (Puertos y Adaptadores)** como estilo principal, implementada como un monolito modular con dos unidades desplegables (API y Worker asíncrono). El núcleo pedagógico (estructura del curso, evaluaciones, cálculo de notas y reglas de aprobación) queda aislado de PostgreSQL, del sistema legado y de la API de plagio. Para la concurrencia en exámenes y las integraciones se complementa con comunicación asíncrona mediante cola de mensajes. La justificación formal se encuentra en el registro de decisiones (sección 3.3).

3.1 MODELO 4+1

3.1.1 VISTA DE ESCENARIOS

La vista de escenarios refleja lo que hace el sistema mediante los casos de uso; cada uno de ellos es realizado por las demás vistas.

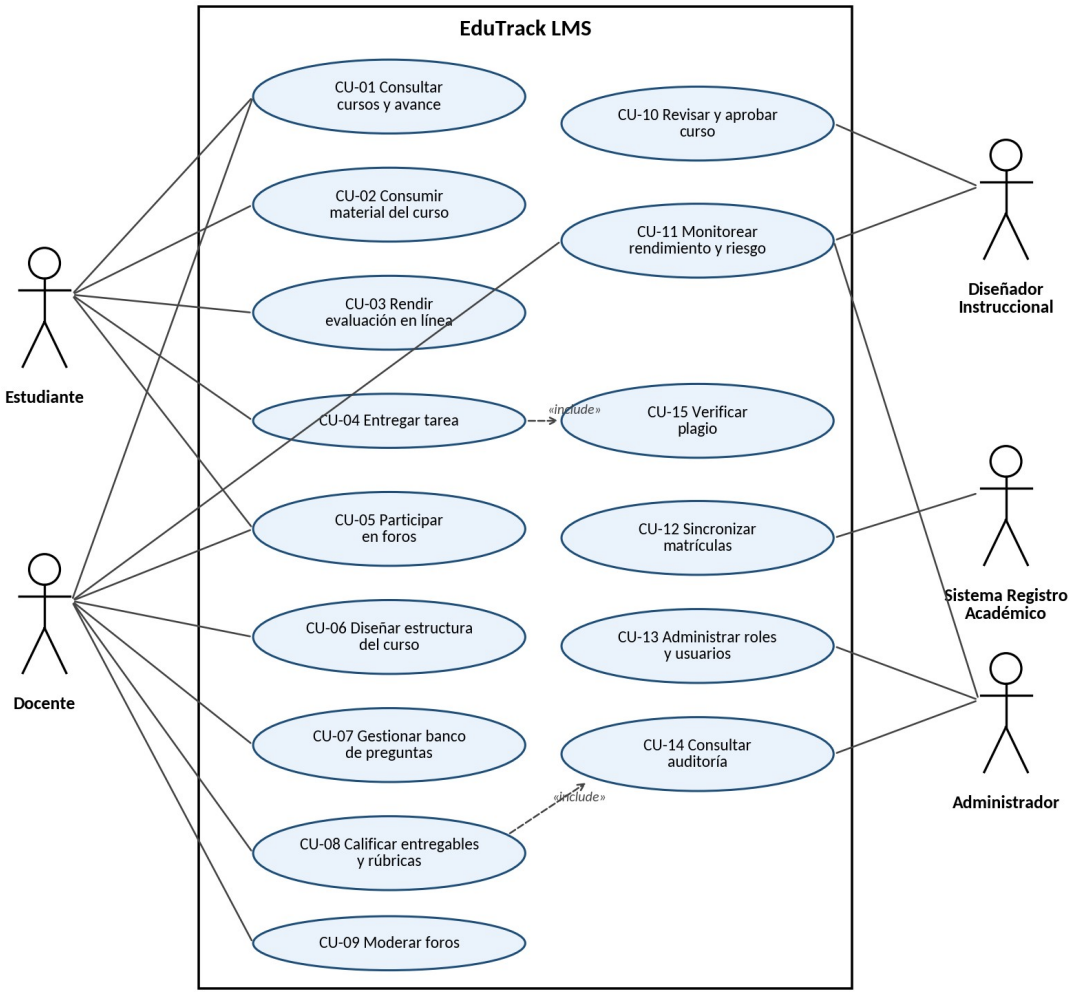


Figura 1. *Diagrama de casos de uso de EduTrack LMS*

Tabla 10. *Especificación del caso de uso CU-03 Rendir evaluación en línea*

Campo	Descripción
Actor	Estudiante
Precondiciones	El estudiante está autenticado y matriculado en el curso; la evaluación está publicada y dentro de su ventana de disponibilidad.
Flujo principal	1. El estudiante inicia el intento y el sistema fija la hora de vencimiento y sortea las preguntas del banco. 2. El estudiante responde; la aplicación autoguarda cada respuesta (máximo cada 15 s). 3. El sistema valida que el intento esté vigente y registra la respuesta de forma idempotente. 4. El estudiante finaliza el intento; el sistema consolida las respuestas y publica el evento Intento Finalizado. 5. El sistema califica automáticamente las preguntas objetivas y registra la nota con su auditoría.
Flujos alternos	A1. Se pierde la conexión: la aplicación conserva las respuestas no confirmadas y las reenvía al reconectar; el servidor descarta duplicados. A2. Vence el tiempo: el sistema cierra el intento automáticamente con las respuestas guardadas.
Postcondiciones	El intento queda finalizado y calificado; no se perdió ninguna respuesta confirmada.

3.1.2 VISTA LÓGICA

La vista lógica presenta las abstracciones clave del dominio y relaciona los requisitos con entidades. La estructura del curso se modela con el patrón Composite y su ciclo de vida con el patrón State.

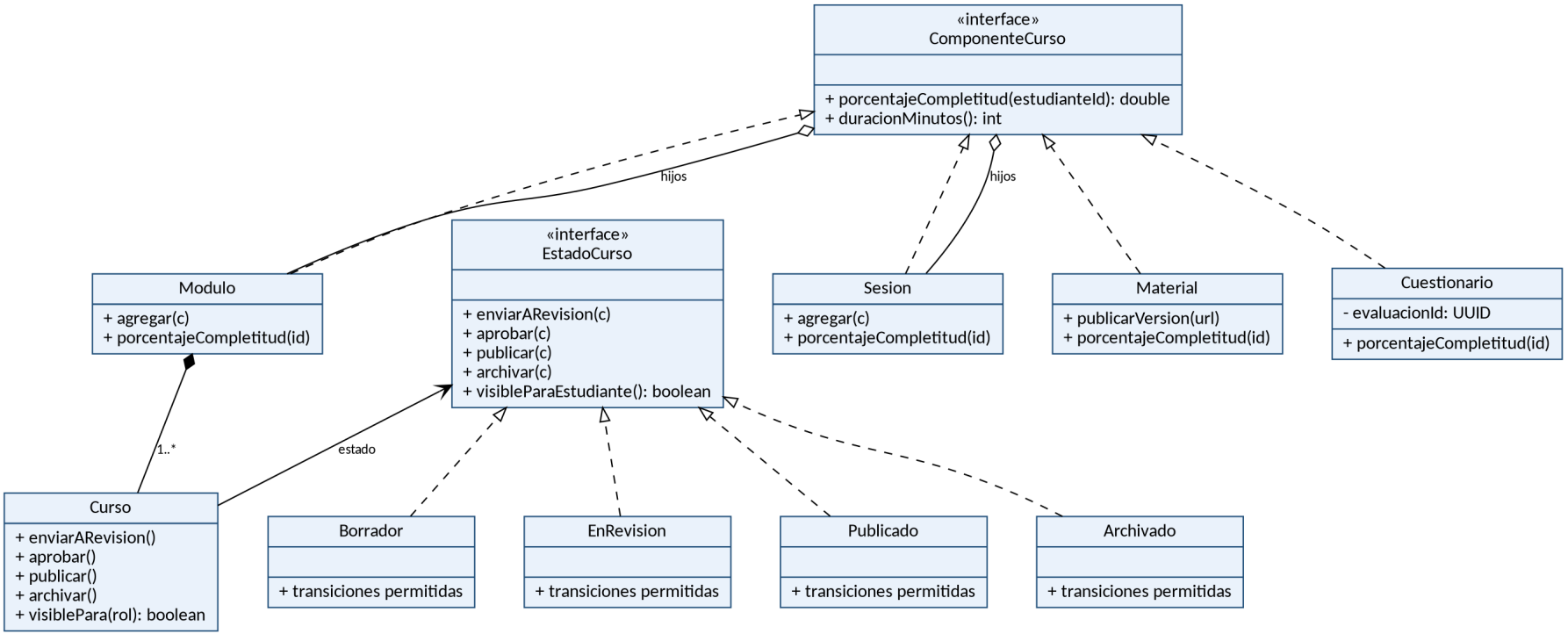


Figura 2. *Diagrama de clases: estructura del curso (Composite) y ciclo de vida (State)*

El motor de evaluaciones utiliza el patrón Strategy para que cada tipo de evaluación (opción múltiple, tarea con rúbrica, foro calificado) aplique su propio algoritmo de calificación sin condicionales en la clase Evaluacion. Las calificaciones son inmutables y cada cambio genera un registro de auditoría.

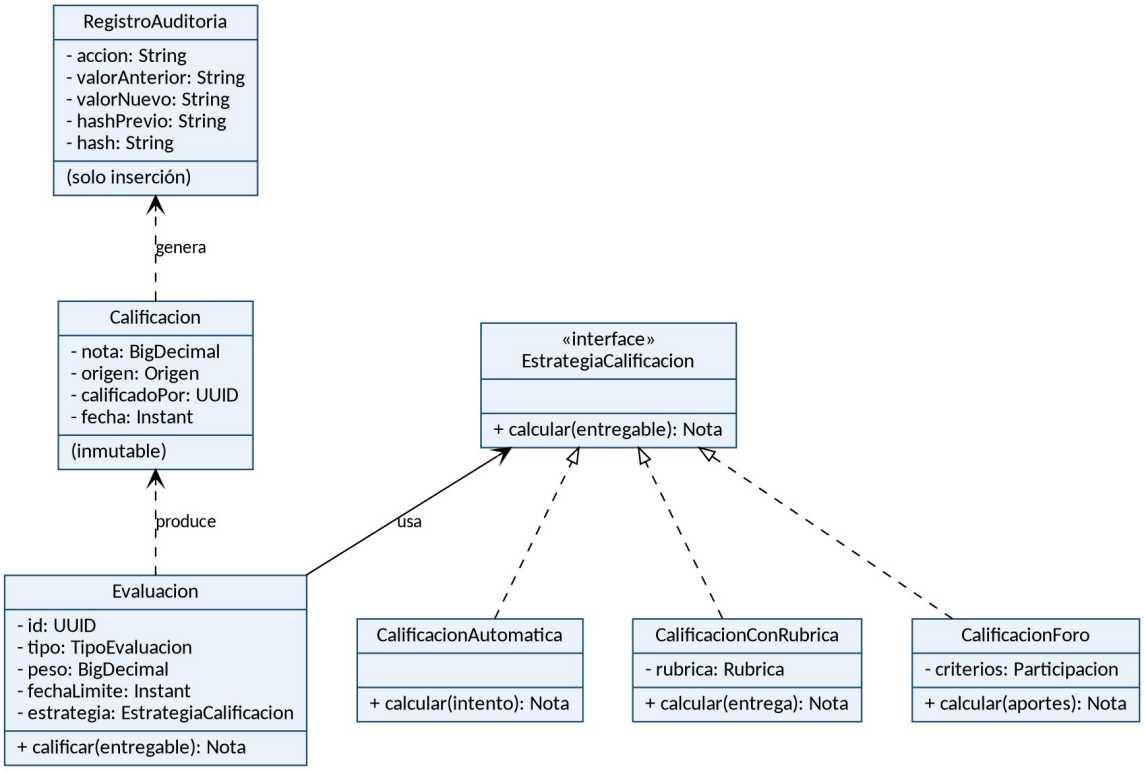


Figura 3. *Diagrama de clases: evaluaciones, calificaciones y auditoría (Strategy)*

Los exámenes concurrentes se modelan con un intento de examen que decide por sí mismo su vigencia y registra respuestas versionadas, lo que hace la operación idempotente.

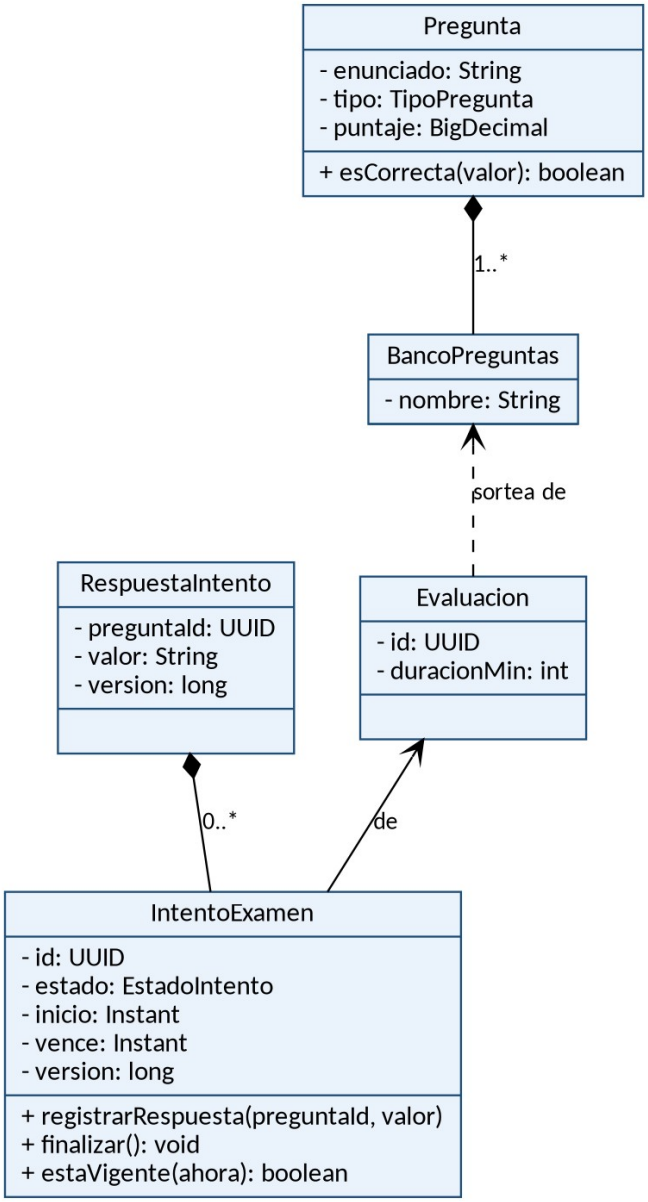


Figura 4. *Diagrama de clases: intento de examen y banco de preguntas*

El foro de discusión aplica el patrón Observer: el hilo notifica a sus suscriptores cuando se publica un mensaje, sin acoplarse a la mensajería.

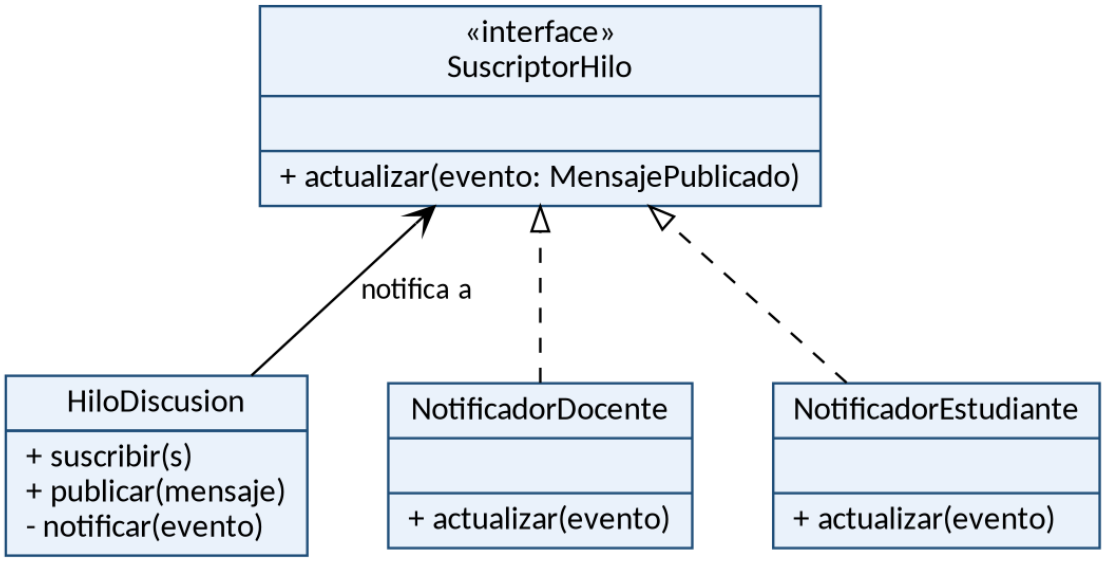


Figura 5. *Diagrama de clases: foro de discusión (Observer)*

Tabla 11. *Trazabilidad entre características y entidades de la vista lógica*

Característica	Entidades y patrones principales
CAR-01, CAR-02	Curso, Modulo, Sesion, Material, VersionMaterial (Composite)
CAR-03	EstadoCurso, Borrador, EnRevision, Publicado, Archivado (State)
CAR-04, CAR-11	BancoPreguntas, Pregunta, Evaluacion, IntentoExamen, RespuestaIntento
CAR-06	EstrategiaCalificacion, CalificacionAutomatica, CalificacionConRubrica, CalificacionForo, Calificacion (Strategy)
CAR-07	HiloDiscusion, SuscriptorHilo, NotificadorDocente, NotificadorEstudiante (Observer)
CAR-09, CAR-10	Puertos RegistroAcademicoPort y PlagioPort con sus adaptadores (Adapter)
CAR-12	RegistroAuditoria (solo inserción, con hash encadenado)

3.1.3 VISTA DE PROCESOS

La vista de procesos muestra cómo, en tiempo de ejecución, el sistema se compone de procesos en interacción. Se presentan los flujos críticos: el examen en línea, la entrega de tareas con verificación de plagio, la publicación en foros, la sincronización de matrículas y el ciclo de vida del curso.

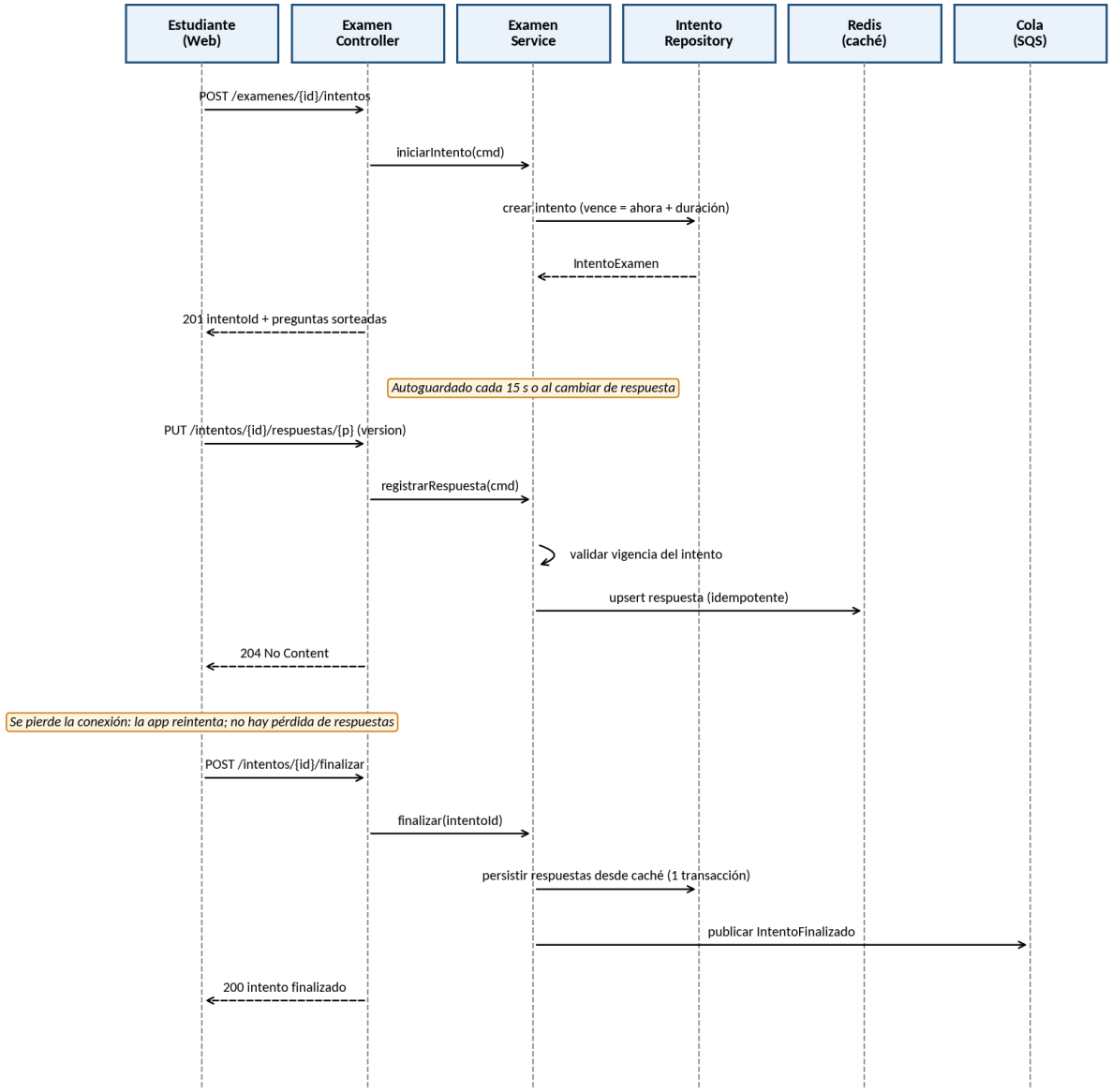


Figura 6. *Diagrama de secuencia: rendir examen con autoguardado idempotente (CU-03)*

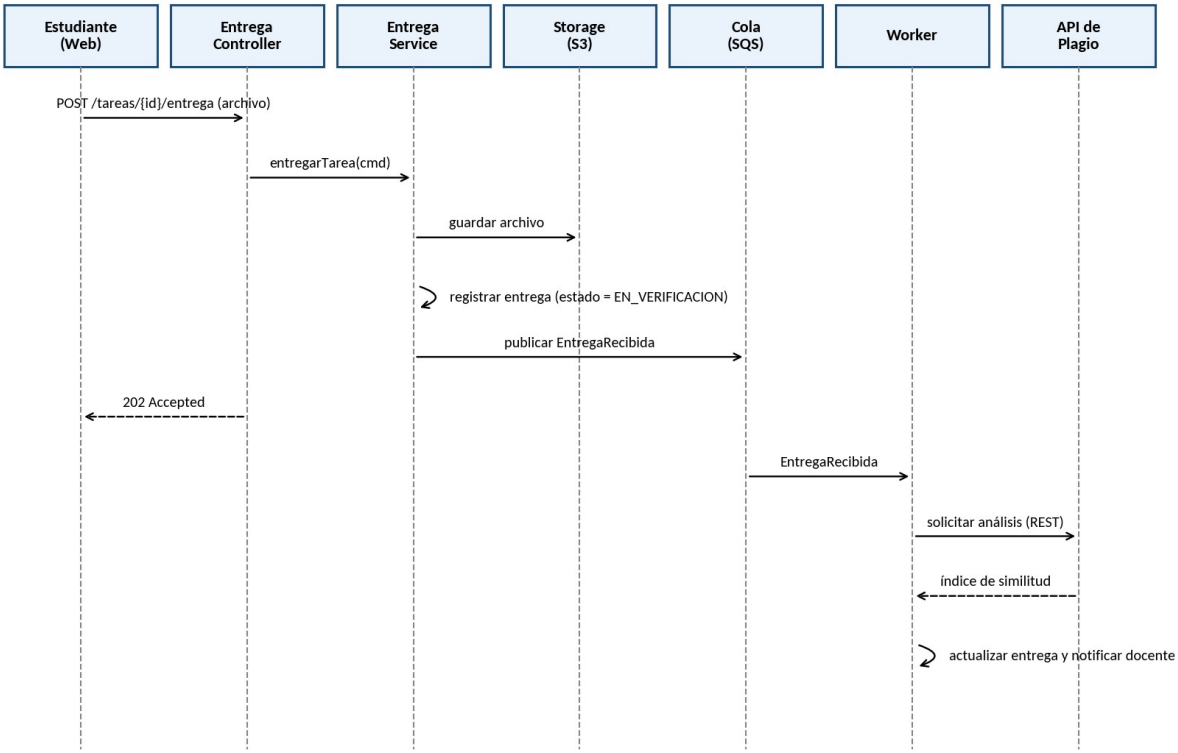


Figura 7. *Diagrama de secuencia: entrega de tarea y verificación asíncrona de plagio (CU-04, CU-15)*

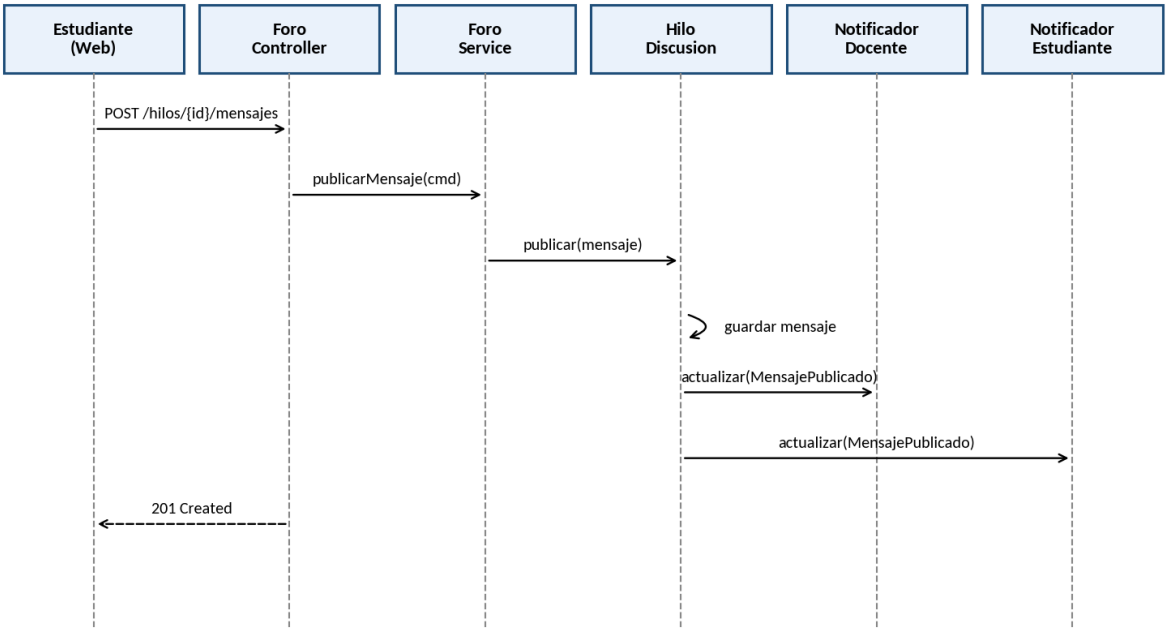


Figura 8. *Diagrama de secuencia: publicación de un mensaje en el foro (CU-05)*

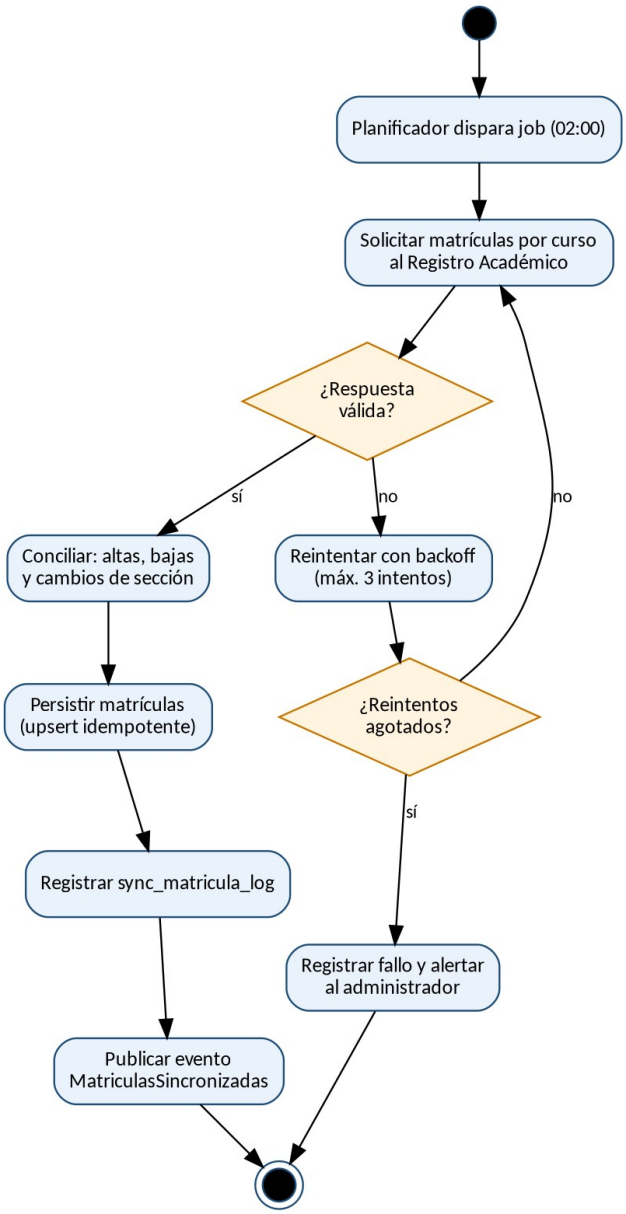


Figura 9. *Diagrama de actividades: sincronización nocturna de matrículas (CU-12)*

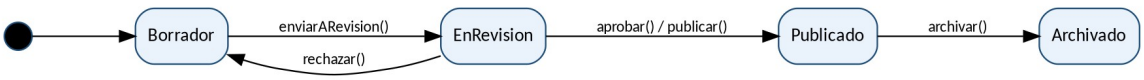


Figura 10. *Diagrama de estados: ciclo de vida del curso (State)*

3.1.4 VISTA DE DESARROLLO

La vista de desarrollo muestra cómo el software se descompone para su construcción. El código se organiza en paquetes que reflejan el hexágono: el dominio y los casos de uso no dependen de ningún framework de infraestructura; los adaptadores de entrada y salida dependen del núcleo y no al revés.

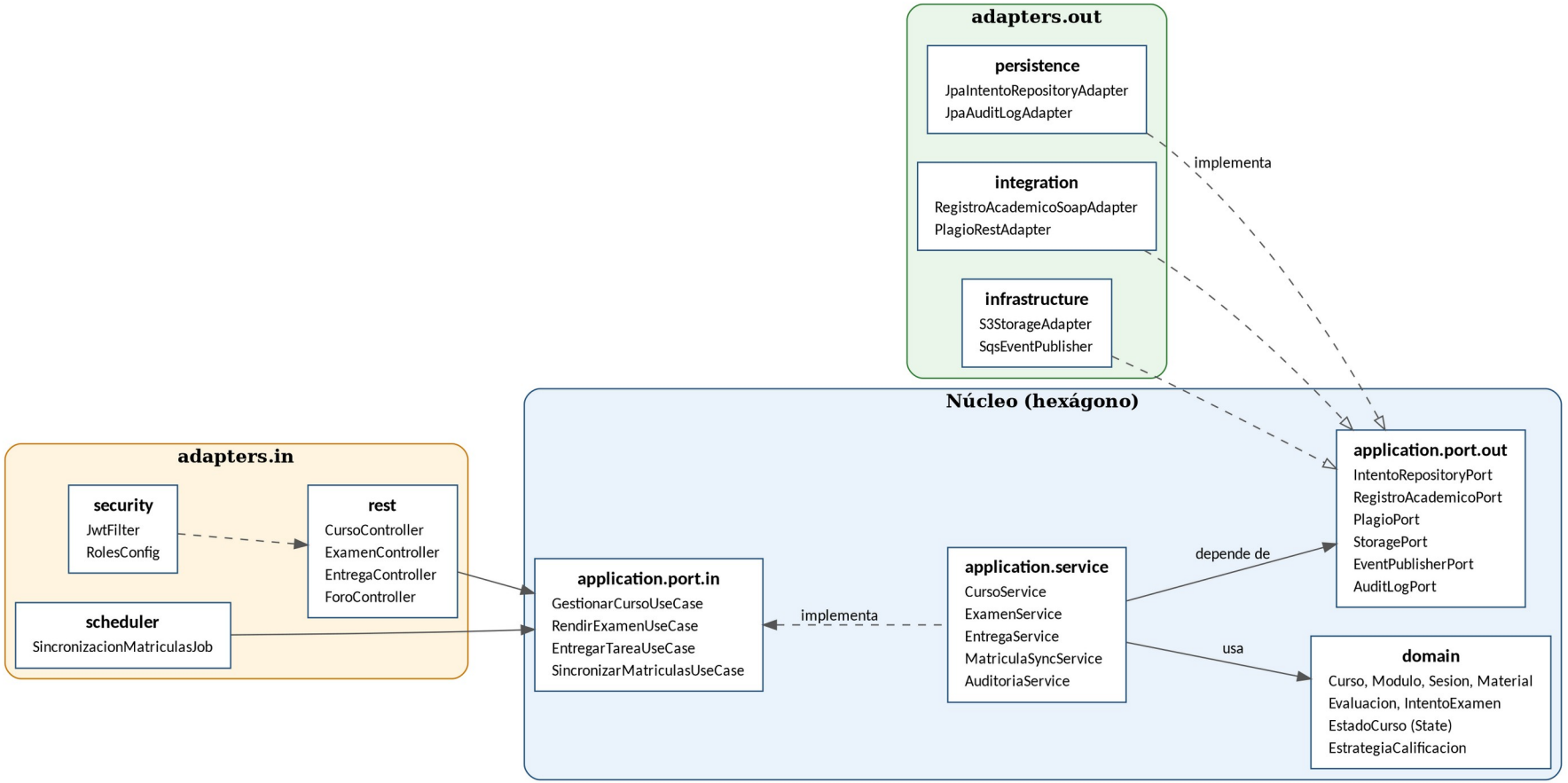


Figura 11. *Diagrama de paquetes de la arquitectura hexagonal*

La estructura propuesta del repositorio es la siguiente:

```
edutrack-lms/
|-- docs/                    (diagramas C4 y 4+1, ADRs, informe)
|-- backend/
|   |-- src/main/java/pe/usmp/edutrack/
|   |   |-- domain/          (model, state, strategy, event)
|   |   |-- application/
|   |   |   |-- port/in/     (casos de uso)
|   |   |   |-- port/out/    (repositorios, plagio, registro, storage, eventos)
|   |   |   `-- service/     (implementación de los casos de uso)
|   |   `-- adapters/
|   |       |-- in/          (rest, scheduler, security)
|   |       `-- out/         (persistence, integration, infrastructure)
|   `-- src/test/java/       (unitarias, integración, ArchUnit)
|-- frontend/                (HTML, CSS y JavaScript)
|-- infra/                   (Dockerfile, IaC, pipelines)
`-- .github/workflows/       (integración y despliegue continuos)
```

Tabla 12. *Reglas de dependencia verificadas con pruebas de arquitectura*

Paquete	Puede depender de	No puede depender de
domain	Solo de la biblioteca estándar de Java	Spring, JPA, SDK de AWS, adaptadores
application	domain	adapters, frameworks de infraestructura
adapters.in	application.port.in, DTOs	adapters.out, domain directamente
adapters.out	application.port.out, domain	adapters.in

3.1.5 VISTA FÍSICA

La vista física describe el despliegue en la nube. La aplicación se ejecuta en contenedores sobre AWS: la capa estática se sirve desde S3 con CloudFront; la API y el Worker corren como tareas independientes que escalan de forma horizontal; PostgreSQL opera en configuración Multi-AZ; y las integraciones externas se invocan desde el Worker.

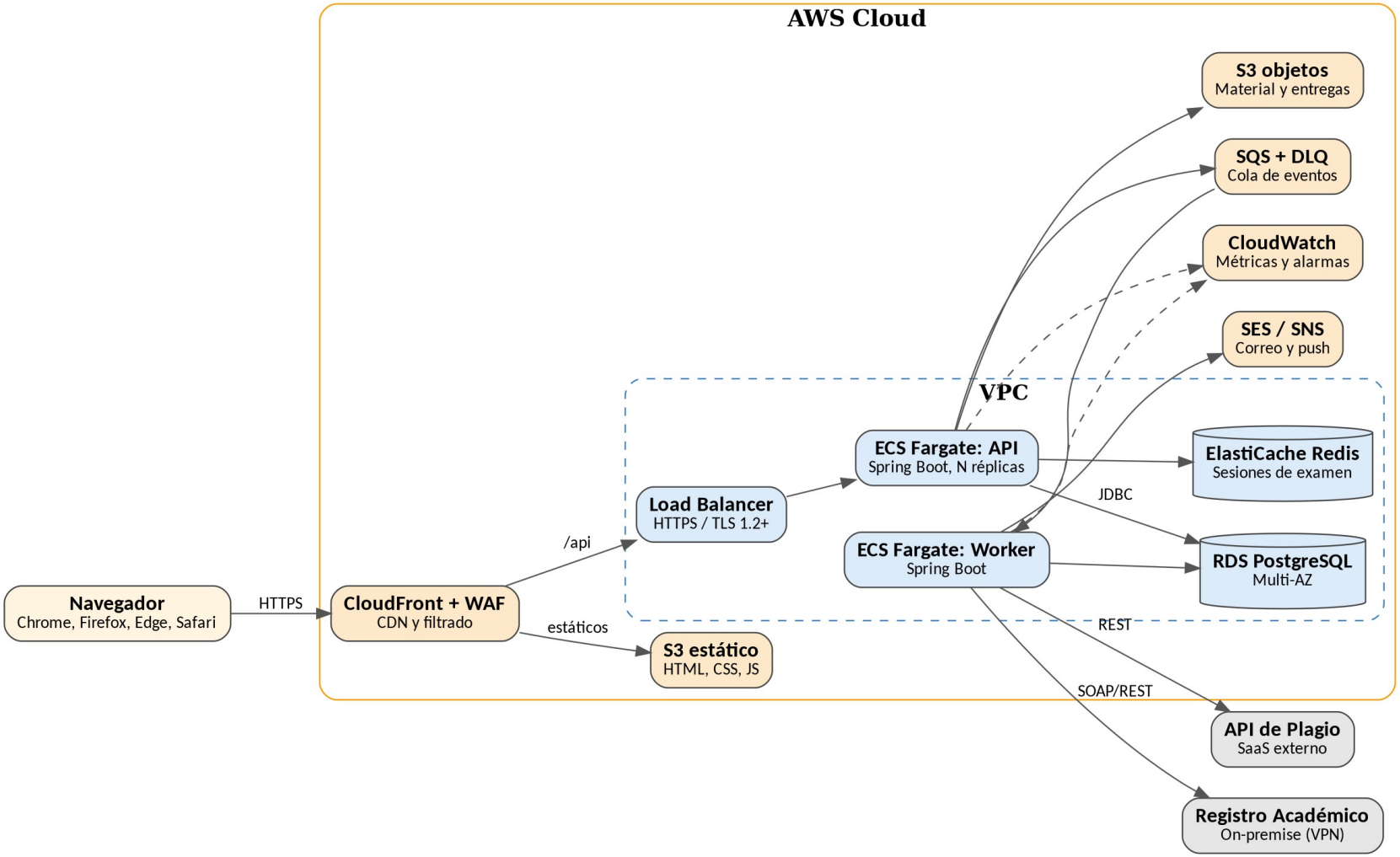


Figura 12. *Diagrama de despliegue en AWS*

Tabla 13. *Nodos de despliegue y tácticas asociadas a los atributos de calidad*

Nodo	Función	Atributo de calidad
CloudFront + WAF	Distribución de contenido estático y filtrado de tráfico malicioso.	Seguridad, desempeño
Application Load Balancer	Balanceo entre réplicas de la API con verificaciones de salud y TLS 1.2+.	Disponibilidad, seguridad
ECS Fargate (API)	Casos de uso expuestos como API REST; autoescalado según CPU y solicitudes.	Desempeño, disponibilidad
ECS Fargate (Worker)	Consume eventos: plagio, notificaciones, analítica y sincronización.	Modificabilidad, interoperabilidad
RDS PostgreSQL Multi-AZ	Datos transaccionales, auditoría inmutable y réplica de lectura para analítica.	Disponibilidad, trazabilidad
ElastiCache Redis	Sesiones de examen y buffer de respuestas parciales.	Desempeño, tolerancia a fallos
S3 y SQS	Almacenamiento versionado de material y entregas; cola de eventos con cola de mensajes fallidos (DLQ).	Disponibilidad, interoperabilidad
CloudWatch	Métricas, registros y alarmas para el monitoreo del rendimiento.	Disponibilidad (CAR-14)

3.2 MODELO C4

3.2.1 DIAGRAMA DE CONTEXTO

El diagrama de contexto muestra EduTrack LMS como una caja única y su relación con las personas que lo utilizan y con los sistemas externos con los que interactúa.

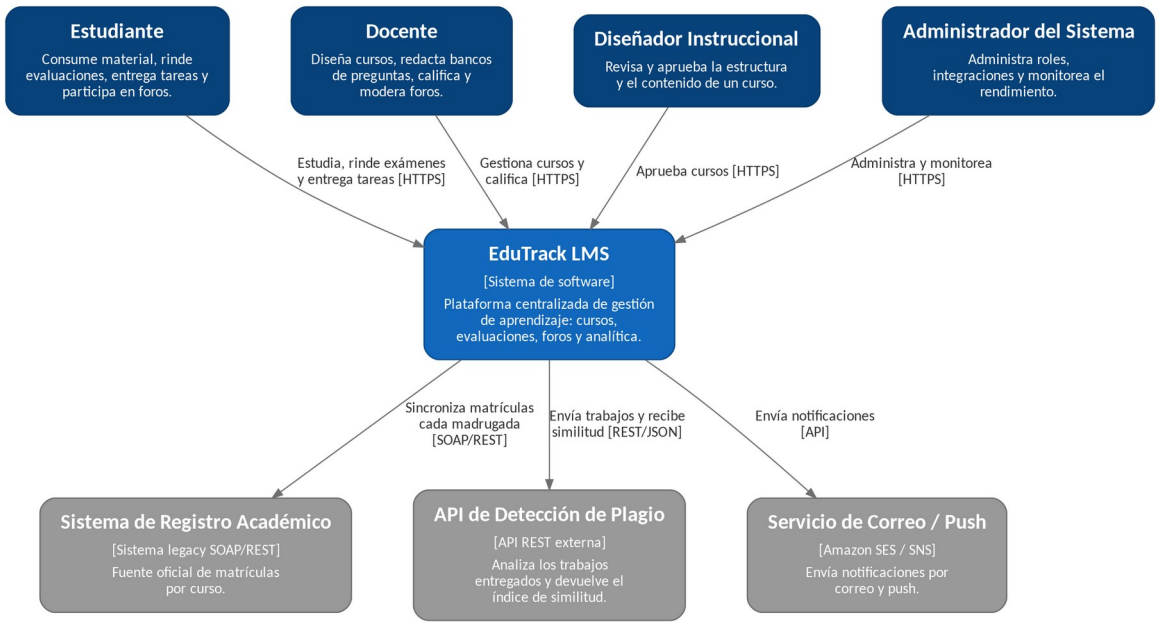


Figura 13. *Diagrama de contexto (C4 nivel 1)*

3.2.2 DIAGRAMA DE CONTENEDORES

El diagrama de contenedores amplía el sistema y muestra las aplicaciones y almacenes de datos que lo componen. El núcleo está en la API Backend; las tareas lentas o dependientes de terceros se delegan al Worker mediante la cola de mensajes, lo que protege el rendimiento de la API en los periodos de exámenes.

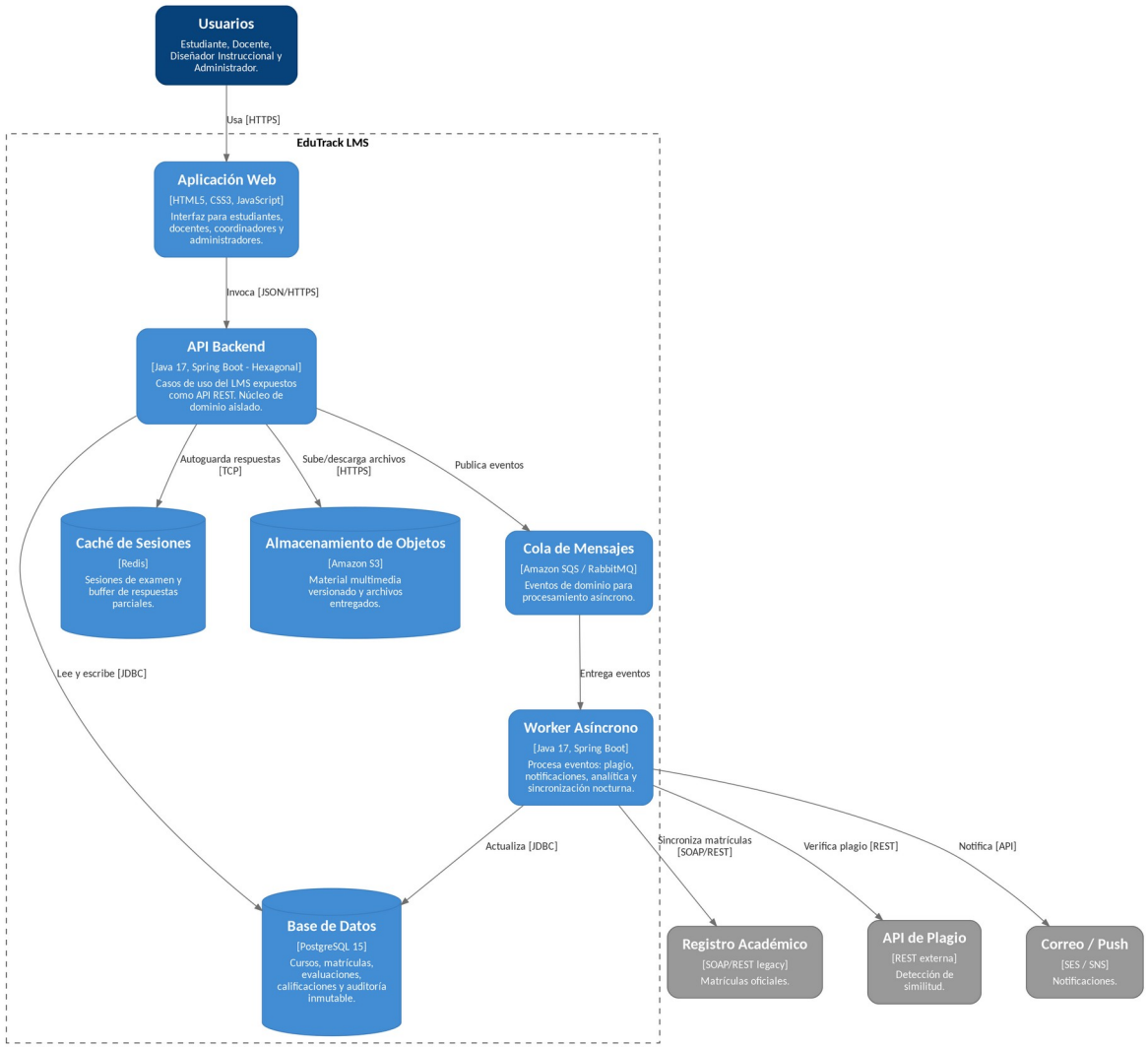


Figura 14. *Diagrama de contenedores (C4 nivel 2)*

3.2.3 DIAGRAMA DE COMPONENTES

El diagrama de componentes descompone el contenedor API Backend en sus componentes principales. Los controladores REST, la seguridad y el planificador son adaptadores de entrada; los casos de uso forman el núcleo; y los clientes de persistencia, caché, almacenamiento, eventos, registro académico y plagio son adaptadores de salida.

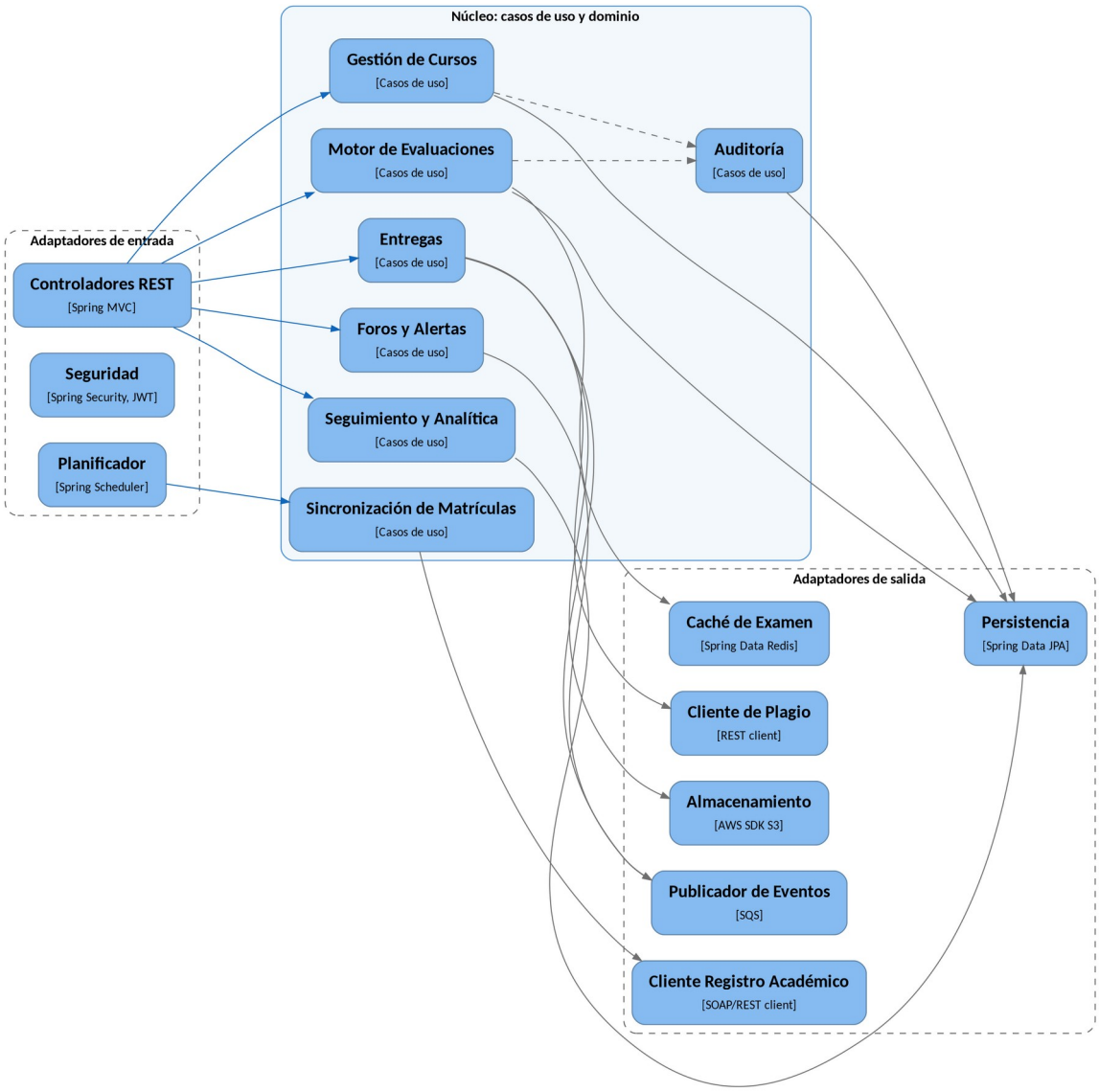


Figura 15. *Diagrama de componentes del contenedor API Backend (C4 nivel 3)*

Tabla 14. *Responsabilidad de los componentes principales*

Componente	Responsabilidad
Gestión de Cursos	Estructura del curso (Composite), versionado de material y ciclo de vida con State.
Motor de Evaluaciones	Banco de preguntas, intentos de examen, calificación (Strategy) y promedios ponderados.
Entregas	Recepción de tareas, almacenamiento y solicitud de verificación de plagio.
Foros y Alertas	Hilos de debate y notificaciones mediante Observer.
Seguimiento y Analítica	Porcentaje de completitud, tiempos de conexión y reglas de riesgo.
Sincronización de Matrículas	Concilia altas, bajas y cambios con el legado de forma idempotente.
Auditoría	Registro inmutable de cambios de calificación y de modificaciones de exámenes publicados.

3.2.4 DISEÑO DE LA BASE DE DATOS

3.2.4.1 Modelo Entidad Relación

El modelo relacional se implementa en PostgreSQL y se presenta en cuatro bloques: académico, evaluaciones y exámenes, entregas, calificaciones y auditoría, y colaboración.

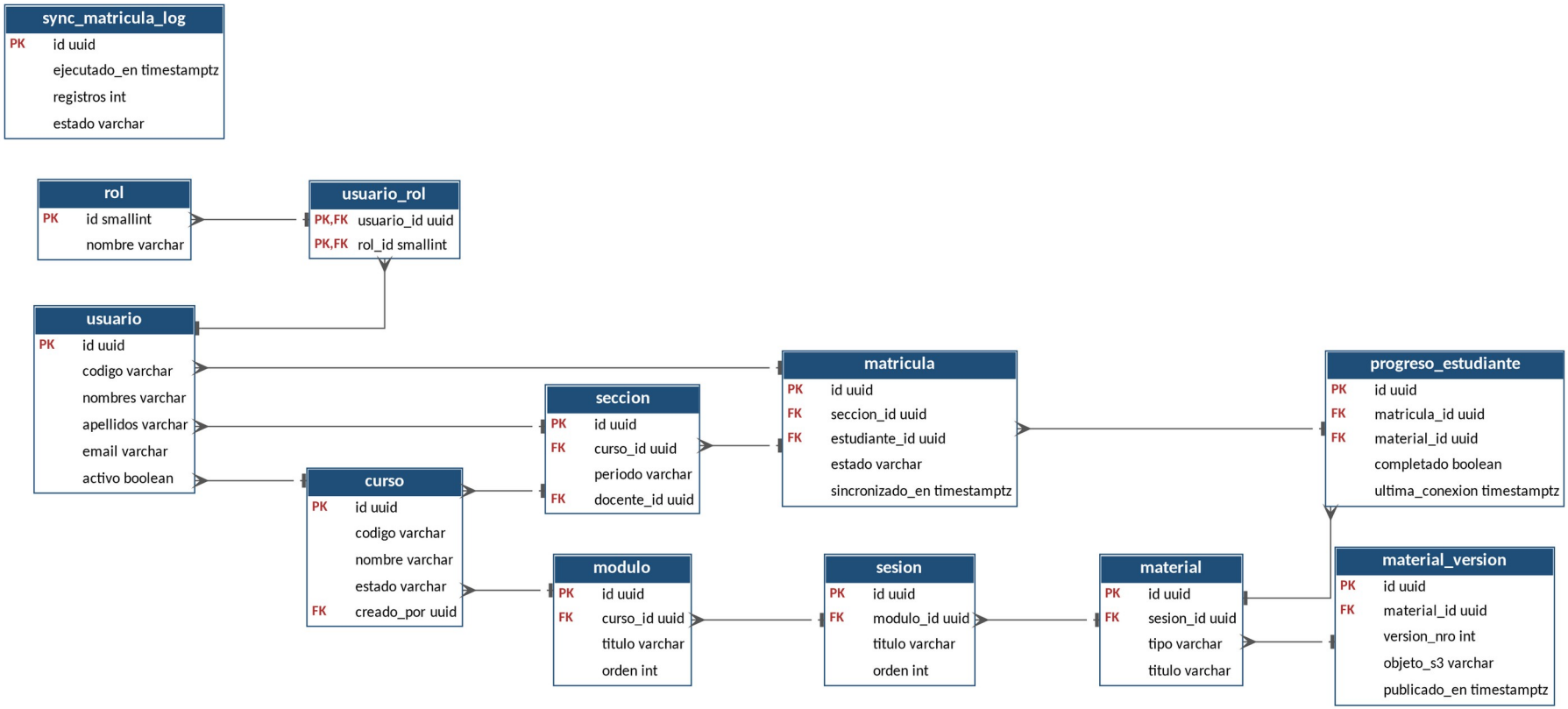


Figura 16. *Modelo entidad relación: bloque académico (usuarios, cursos, matrículas y contenido)*

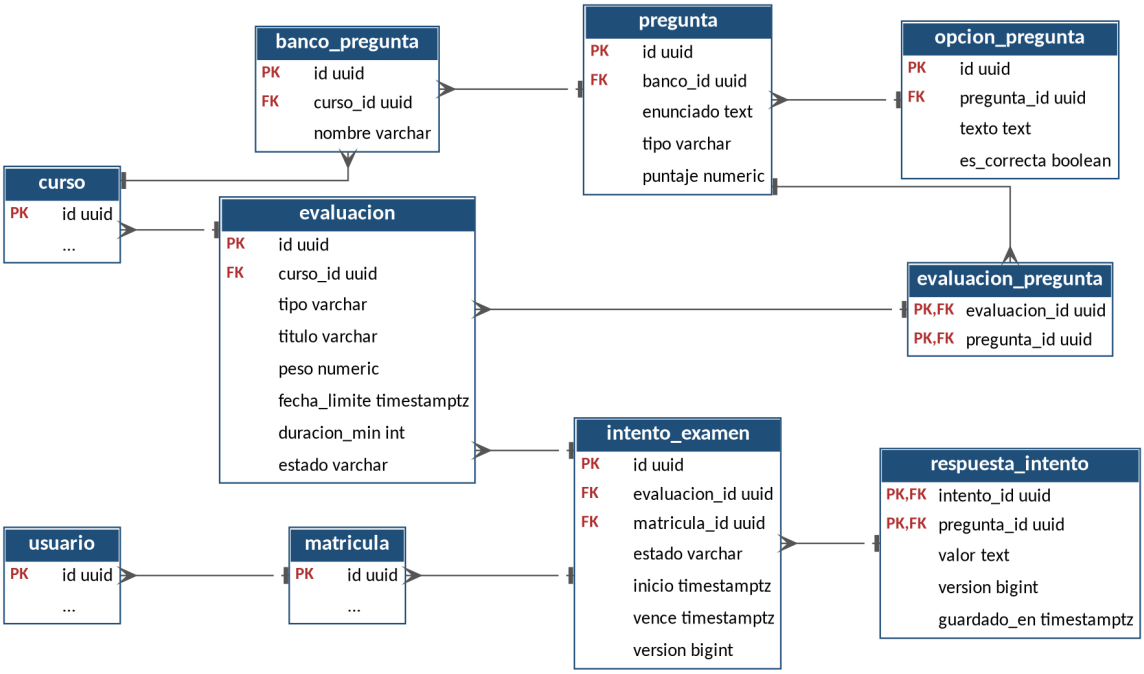


Figura 17. *Modelo entidad relación: banco de preguntas, evaluaciones e intentos de examen*

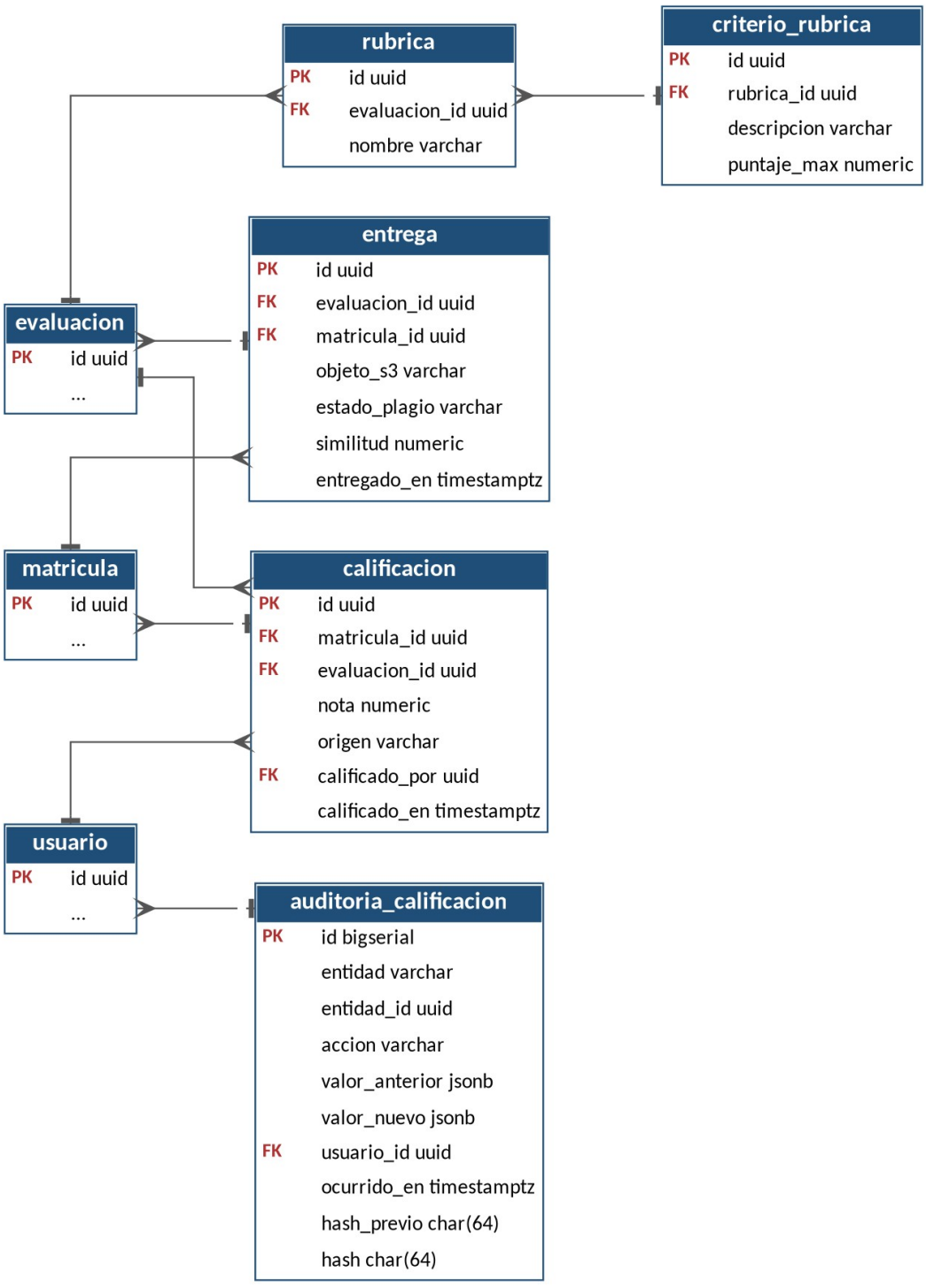


Figura 18. *Modelo entidad relación: entregas, rúbricas, calificaciones y auditoría*

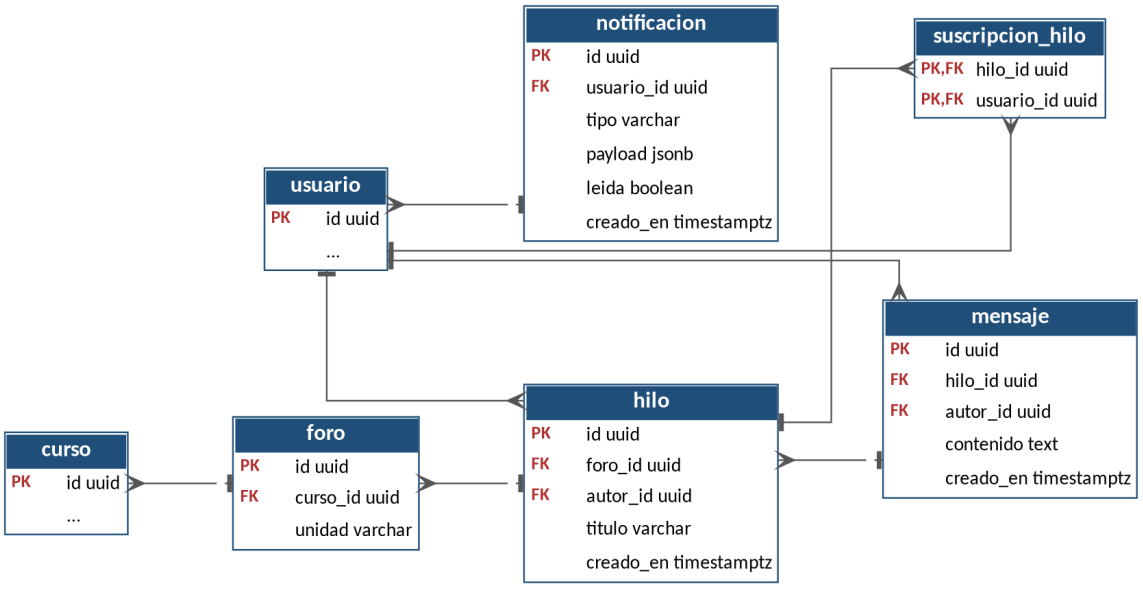


Figura 19. *Modelo entidad relación: foros y notificaciones*

3.2.4.2 Decisiones de diseño relevantes

- **Concurrencia sin bloqueos:** la tabla respuesta_intento tiene clave primaria compuesta (intento_id, pregunta_id), por lo que los estudiantes escriben filas distintas y no compiten entre sí; la columna version permite el control optimista y la idempotencia.
- **Auditoría inmutable:** la tabla auditoria_calificacion es de solo inserción; cada fila guarda el hash SHA-256 de la fila anterior (hash_previo), de modo que cualquier alteración rompe la cadena y es detectable.
- **Sincronización idempotente:** la tabla matricula se actualiza mediante inserciones con resolución de conflictos y se registra cada ejecución en sync_matricula_log.
- **Material versionado:** material_version conserva cada versión publicada; el archivo reside en S3 y la base de datos guarda solo la referencia.

```
REVOKE UPDATE, DELETE ON auditoria_calificacion FROM edutrack_app;

CREATE FUNCTION bloquear_modificacion() RETURNS trigger AS $$
BEGIN
  RAISE EXCEPTION 'El registro de auditoría es inmutable';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auditoria_inmutable
  BEFORE UPDATE OR DELETE ON auditoria_calificacion
  FOR EACH ROW EXECUTE FUNCTION bloquear_modificacion();
```

3.3 REGISTRO DE DECISIONES ARQUITECTÓNICAS (ADR)

Cada decisión se documenta con el formato propuesto por Nygard (2011): contexto, decisión, alternativas consideradas y consecuencias.

Tabla 15. *ADR-001. Arquitectura Hexagonal como estilo principal*

Estado	Aceptada
Contexto	El núcleo pedagógico (cálculo de notas, aprobación, estructura del curso) debe permanecer estable aunque cambien la base de datos, el sistema legado o el proveedor de plagio (EAC-07, EAC-08, RES-01, RES-02).
Decisión	Se adopta Arquitectura Hexagonal: el dominio y los casos de uso definen puertos; REST, planificador, JPA, S3 y clientes externos son adaptadores.
Alternativas	Arquitectura en capas estricta (Presentación, Negocio, Acceso a datos); microservicios por módulo.
Consecuencias	Positivas: pruebas unitarias del dominio sin PostgreSQL; reemplazo de integraciones sin tocar el núcleo. Negativas: mayor cantidad de interfaces y mapeos entre DTO, dominio y entidades.

Tabla 16. *ADR-002. Monolito modular con dos unidades desplegables*

Estado	Aceptada
Contexto	El equipo es reducido y el plazo es académico, pero el sistema debe escalar en exámenes (EAC-01) y aislar tareas lentas (plagio, notificaciones).
Decisión	Un único código base modular con dos despliegues: API (síncrona) y Worker (asíncrono), que comparten el dominio.
Alternativas	Microservicios independientes; monolito con un único despliegue.
Consecuencias	Positivas: menor complejidad operativa y posibilidad de escalar la API de forma independiente. Negativas: los módulos comparten esquema de base de datos y requieren disciplina para respetar los límites (ArchUnit).

Tabla 17. *ADR-003. Estrategia de concurrencia para exámenes en línea*

Estado	Aceptada
Contexto	Se esperan picos masivos de escritura sin bloqueo de base de datos y sin pérdida de respuestas ante desconexiones (CAR-11, EAC-01, EAC-03).
Decisión	El cliente autoguarda cada respuesta con un número de versión. El servidor valida la vigencia del intento y registra la respuesta de forma idempotente en Redis (con persistencia habilitada); un proceso de respaldo periódico y la finalización del intento consolidan los datos en PostgreSQL mediante inserciones con resolución de conflictos. El cliente conserva localmente las respuestas no confirmadas y las reenvía al reconectar.
Alternativas	Escribir cada respuesta de forma síncrona en PostgreSQL; bloqueo pesimista del intento; conexiones persistentes con WebSockets.
Consecuencias	Positivas: baja latencia y cero contención entre estudiantes; reintentos seguros. Negativas: consistencia eventual entre Redis y PostgreSQL durante el examen y mayor complejidad de pruebas; se mitiga con pruebas de carga y de caída de Redis.

Tabla 18. *ADR-004. Registro de auditoría inmutable con hash encadenado*

Estado	Aceptada
Contexto	Todo cambio de calificación o de un examen publicado debe ser trazable e inalterable (CAR-12, EAC-04).
Decisión	Tabla de solo inserción con hash SHA-256 encadenado, permisos de actualización y borrado revocados y un disparador que bloquea cualquier modificación.

Alternativas	Event Sourcing completo; servicio de registro externo; almacenamiento en cadena de bloques.
Consecuencias	Positivas: inmutabilidad verificable con bajo costo operativo. Negativas: el administrador de base de datos con privilegios elevados sigue siendo un punto de confianza; se mitiga con réplica de solo lectura y verificación periódica de la cadena.

Tabla 19. *ADR-005. Integración mediante puertos, adaptadores y sincronización idempotente*

Estado	Aceptada
Contexto	El legado de matrículas es un servicio SOAP o REST propio de la institución y la API de plagio es de un tercero (RES-01, RES-02, EAC-06).
Decisión	Cada integración se expone al dominio como un puerto de salida (RegistroAcademicoPort, PlagioPort) con un adaptador que traduce formatos y protege al núcleo (capa anticorrupción). La sincronización nocturna usa reintentos con retroceso y actualizaciones idempotentes.
Alternativas	Invocar el legado directamente desde los servicios de aplicación; replicar la base de datos del legado.
Consecuencias	Positivas: fallos de terceros aislados y proveedores intercambiables. Negativas: se debe mantener la traducción de modelos y los contratos de las integraciones.

Tabla 20. *ADR-006. Comunicación asíncrona mediante cola de mensajes*

Estado	Aceptada
Contexto	La verificación de plagio, las notificaciones y la analítica no deben afectar la latencia de la API durante los exámenes (EAC-01, EAC-06).
Decisión	La API publica eventos de dominio en una cola (Amazon SQS) con cola de mensajes fallidos; el Worker los consume y ejecuta las tareas lentas.
Alternativas	Procesamiento síncrono dentro de la solicitud; eventos internos en memoria.
Consecuencias	Positivas: desacoplamiento y absorción de picos. Negativas: consistencia eventual y necesidad de monitorear la cola.

Tabla 21. *ADR-007. Despliegue en AWS con contenedores y servicios administrados*

Estado	Aceptada
Contexto	Se requiere alta disponibilidad en periodos críticos y uso de tecnologías en la nube (RES-07, EAC-02).
Decisión	Contenedores en ECS Fargate detrás de un Application Load Balancer, RDS PostgreSQL Multi-AZ, ElastiCache Redis, S3, SQS y CloudFront, con despliegue continuo mediante GitHub Actions.
Alternativas	Servidores propios (on-premise); máquinas virtuales administradas manualmente; otro proveedor de nube.
Consecuencias	Positivas: autoescalado y recuperación rápida. Negativas: costo variable y dependencia del proveedor; se mitiga con contenedores portables e infraestructura como código.

3.4 PATRONES DE DISEÑO Y PRINCIPIOS APLICADOS

Tabla 22. *Patrones de diseño aplicados al dominio*

Patrón	Dónde se aplica	Problema que resuelve
Composite (GoF)	Curso, Módulo, Sesión, Material y Cuestionario.	Trata de manera uniforme agrupaciones y elementos individuales, por ejemplo al calcular el porcentaje de completitud del curso.
Strategy (GoF)	Algoritmos de calificación por tipo de evaluación.	Evita condicionales en Evaluacion; se inyecta el algoritmo correcto en tiempo de ejecución.
State (GoF)	Ciclo de vida del curso (Borrador, En revisión, Publicado, Archivado).	Encapsula el comportamiento y las transiciones permitidas de cada fase.
Observer (GoF)	Hilos de foro y alertas.	Desacopla la publicación de un mensaje de la lógica de notificación.
Adapter (GoF)	Clientes del Registro Académico, API de plagio, S3 y SQS.	Traduce protocolos y formatos externos a los puertos del dominio.
Repository	Puertos de persistencia con adaptadores JPA.	Oculta el acceso a datos al núcleo de dominio.

Tabla 23. *Principios SOLID y patrones GRASP aplicados*

Principio	Aplicación en EduTrack LMS
SRP	Cada servicio de aplicación implementa un caso de uso; los DTO evitan filtrar entidades de base de datos a los controladores.
OCP	Un nuevo tipo de evaluación se agrega con una nueva estrategia, sin modificar Evaluacion.
LSP	Todas las implementaciones de EstrategiaCalificacion y EstadoCurso son sustituibles sin alterar el comportamiento esperado.
ISP	Los puertos son pequeños y específicos por caso de uso (por ejemplo PlagioPort y StoragePort).
DIP	Los servicios dependen de los puertos (abstracciones), no de JPA, S3 ni de clientes concretos.
GRASP	Experto en información (IntentoExamen decide su vigencia), Controlador, Bajo acoplamiento y alta cohesión, Polimorfismo (Strategy), Variaciones protegidas (puertos) y Fabricación pura (AuditoriaService).

Como evidencia de la aplicación de estos patrones, se muestran fragmentos de la prueba de concepto: la estrategia de calificación y el registro idempotente de respuestas en el intento de examen.

```
public interface EstrategiaCalificacion {
    Nota calcular(Entrega entrega);
}

public class Evaluacion {
    private final EstrategiaCalificacion estrategia;

    public Nota calificar(Entrega entrega) {
        return estrategia.calcular(entrega);
    }
}
```

```
public void registrarRespuesta(UUID preguntaId, String valor,
                               long versionCliente, Instant ahora) {
```

```
    if (!estaVigente(ahora)) {
        throw new IntentoVencidoException(id);
    }
    RespuestaIntento actual = respuestas.get(preguntaId);
    if (actual != null && versionCliente <= actual.version()) {
        return; // reintento duplicado: la operación es idempotente
    }
    respuestas.put(preguntaId,
        new RespuestaIntento(preguntaId, valor, versionCliente));
}
```

Entre los antipatrones que se busca evitar se encuentran las clases con demasiadas responsabilidades (God Class), el modelo de dominio anémico, el acceso directo a datos desde los controladores y los condicionales por tipo de evaluación.

3.5 PRUEBA DE CONCEPTO, PRUEBAS Y GESTIÓN DEL PROYECTO

3.5.1 ALCANCE DE LA PRUEBA DE CONCEPTO

La prueba de concepto implementa en Java con Spring Boot y PostgreSQL los módulos críticos: el **Motor de Evaluaciones** (intentos de examen, calificación con Strategy y auditoría) y la **Integración de Matrículas** (puerto, adaptador y sincronización idempotente). Incluye la estructura de proyecto conforme a la arquitectura hexagonal y un front-end básico en HTML, CSS y JavaScript para rendir un examen.

3.5.2 ESTRATEGIA DE PRUEBAS

Tabla 24. *Estrategia de pruebas y atributos de calidad verificados*

Tipo de prueba	Herramienta	Qué verifica
Unitarias del dominio	JUnit 5 y Mockito	Reglas de calificación, vigencia del intento e idempotencia (EAC-08).
Arquitectura	ArchUnit	Reglas de dependencia del hexágono (ADR-001).
Integración	Testcontainers (PostgreSQL)	Repositorios, restricciones de auditoría inmutable (EAC-04).
Contrato de integraciones	WireMock	Adaptadores del Registro Académico y de plagio con fallos simulados (EAC-06).
Carga y concurrencia	Gatling o JMeter	Autoguardado concurrente de respuestas contra el objetivo de EAC-01.

3.5.3 GESTIÓN CON SCRUM Y GITHUB

El desarrollo sigue la metodología SCRUM en sprints de dos semanas. Los requisitos funcionales y no funcionales se registran como historias de usuario en GitHub Projects, enlazadas a las características (CAR) y a los escenarios de calidad (EAC); los commits evidencian la implementación de la arquitectura y los patrones.

Tabla 25. *Planificación propuesta de sprints*

Sprint	Objetivo	Historias principales
0	Preparación	Repositorio, tablero de GitHub Projects, integración continua y diagramas de arquitectura en el repositorio.
1	Núcleo de cursos	CAR-01, CAR-02, CAR-03: estructura del curso (Composite) y ciclo de vida (State).
2	Motor de evaluaciones	CAR-04, CAR-06: banco de preguntas y calificación con Strategy; adaptadores de persistencia.
3	Exámenes concurrentes y auditoría	CAR-11, CAR-12: autoguardado idempotente, buffer de respuestas y registro inmutable.
4	Integraciones	CAR-09, CAR-10: sincronización de matrículas y verificación de plagio.
5	Cierre	Pruebas de carga, despliegue en la nube, documentación y exposición.

Tabla 26. *Ejemplos de historias de usuario con sus requisitos asociados*

ID	Historia de usuario	Asociado a
HU-01	Como estudiante, quiero rendir un examen contra reloj para que mis respuestas queden guardadas aunque pierda la conexión.	CAR-04, CAR-11, EAC-03
HU-02	Como docente, quiero calificar con rúbricas para registrar notas ponderadas de las tareas.	CAR-06
HU-03	Como administrador, quiero consultar el historial de cambios de notas para auditar modificaciones.	CAR-12, EAC-04
HU-04	Como sistema, quiero sincronizar las matrículas cada madrugada para que las listas de cada curso estén actualizadas.	CAR-09, EAC-06
HU-05	Como diseñador instruccional, quiero aprobar un curso antes de su publicación para asegurar el modelo didáctico.	CAR-03

CONCLUSIONES

1. Se identificaron y priorizaron los drivers arquitectónicos de EduTrack LMS; los más determinantes son la concurrencia en exámenes (EAC-01), la disponibilidad (EAC-02) y la trazabilidad de calificaciones (EAC-04), los cuales guiaron las principales decisiones de diseño.
2. La Arquitectura Hexagonal permite aislar el núcleo pedagógico de PostgreSQL, del sistema legado de matrículas y de la API de plagio, con lo que se cumple el objetivo de facilitar las pruebas del dominio y reemplazar integraciones sin modificar el núcleo.
3. La estrategia de concurrencia propuesta (autoguardado idempotente, buffer en Redis, consolidación en PostgreSQL y procesamiento asíncrono) evita bloqueos de base de datos y protege las respuestas parciales; su cumplimiento debe confirmarse con pruebas de carga.
4. El registro de auditoría de solo inserción con hash encadenado satisface la restricción de trazabilidad inmutable con un costo operativo bajo.
5. El modelado con 4+1 vistas y C4, el registro de ADR y los patrones GoF y principios SOLID documentan la arquitectura de forma consistente y verificable, y establecen la base para la prueba de concepto.

RECOMENDACIONES

6. Validar con la institución las cifras de carga supuestas (estudiantes concurrentes, tamaño de archivos, ventanas de examen) y ajustar el dimensionamiento antes del despliegue.
7. Ejecutar pruebas de carga y de caída de componentes (Redis, instancia de API) para confirmar los escenarios EAC-01, EAC-02 y EAC-03.
8. Evaluar la extracción del Motor de Evaluaciones como servicio independiente si la carga o la organización del equipo lo justifican, aprovechando los puertos ya definidos.
9. Incorporar un panel de analítica con reglas de riesgo configurables y, en una etapa posterior, modelos predictivos de deserción.
10. Definir una política de retención y verificación periódica de la cadena de auditoría, y revisar los permisos de la base de datos con el área de seguridad.

BIBLIOGRAFÍA

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Brown, S. (s. f.). *The C4 model for visualising software architecture*. https://c4model.com

Cervantes, H., Velasco-Elizondo, P., & Castro, L. (2016). *Arquitectura de software: Conceptos y ciclo de desarrollo*. Cengage Learning.

Cockburn, A. (2001). *Writing effective use cases*. Addison-Wesley.

Cockburn, A. (2005). *Hexagonal architecture*. https://alistair.cockburn.us/hexagonal-architecture/

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software, 12*(6), 42-50.

Larman, C. (2004). *Applying UML and patterns: An introduction to object-oriented analysis and design and iterative development* (3.ª ed.). Prentice Hall.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Nygard, M. (2011, 15 de noviembre). *Documenting architecture decisions*. Cognitect. https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions

Schwaber, K., & Sutherland, J. (2020). *La guía de Scrum*. https://scrumguides.org